

SQL i nie tylko

Wykład drugi

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Proszę robić
notatki :-)

Plan na dziś

Plan na dziś

Z tego wykładu dowiedziecie się:

- + o wartości **NULL**
 - + o typach danych
 - + o relacjach „jeden do wielu”
 - + jak zaprojektować relacyjną bazę danych
 - + jak utworzyć bazę danych
 - + jak wypełnić bazę danych danymi
- ...i poznacie pierwsze zapytanie :-)

Przepraszam was za to, że w bazie
danych nie będzie danych o ulubionych
zupach naszych bohaterów :-)

NULL

NULL

W SQL-u NULL oznacza brak danych



NULL

Wyobraźmy sobie bazę danych o gatunkach drzew. Oto jej fragment:

GatunkiDrzew

nazwa	kolor_liści	kolor_igieł
jodła pospolita	NULL	ciemnozielony
sosna zwyczajna	NULL	szarozielony
buk zwyczajny	ciemnozielony	NULL
lipa drobnolistna	żywozielony	NULL

Typy danych

Typy danych

Tak nazywają się typy danych w SQL-u:

- + tekst — TEXT :-)
- + liczba całkowita — INT
- + liczba rzeczywista — REAL

Relacje „jeden do wielu”



Relacje „jeden do wielu”

Wyobraźmy sobie bazę danych o gatunkach zwierząt.
Do jednego rodzaju może należeć wiele gatunków.
Oto fragment tej bazy danych:

Rodzaj

id_rodzaju [KG]	nazwa
123	wilk

Gatunek

id_gatunku [KG]	nazwa	id_rodzaju [K0]
321	wilk szary	123
322	szakał złocisty	123
323	kojot preriowy	123
324	pies domowy	123

Programiści mówią: tabela Rodzaj jest w relacji „jeden do wielu” z tabelą Gatunek

Relacje „jeden do wielu”

Oprócz relacji „jeden do wielu” są też relacje „jeden do jednego” i „wiele do wielu”, ale tych relacji nie widać na pierwszy rzut oka
Poznamy te relacje później :-)

Relacje „jeden do wielu”

Wyobraźmy sobie bazę danych o studentach.
Jeden student może studiować wiele kierunków.
Oto fragment tej bazy danych:

Student

nr_indeksu	imię	nazwisko
22222	Joanna	Jawor
44444	Barbara	Buk

Studia

nr_indeksu	kierunek
22222	bioinformatyka
44444	bioinformatyka
44444	kryminalistyka

Relacje „jeden do wielu”

Podczas projektowania bazy danych dobrze jest pamiętać o zasadzie DRY (Don't Repeat Yourself), czyli „nie powtarzaj się”

Relacje „jeden do wielu”

Gdybyśmy zapisali te same dane w jednej tabeli:

Student

nr_indeksu [KG]	imię	nazwisko	kierunek
22222	Janina	Jawor	bioinformatyka
44444	Barbara	Buk	bioinformatyka
44444	Barbara	Buk	kryminalistyka

to w tej tabeli niepotrzebnie powtarzałyby się dane 44444 Barbara Buk

Relacje „jeden do wielu”

Takie bazy danych, w których niepotrzebnie powtarzają się dane, mają kilka wad:

- + Są większe, niż mogłyby być
- + Wpisując te same dane w różnych wierszach, moglibyśmy się pomylić. Wtedy system zarządzania bazą danych nie wiedziałby, że te dane opisują to samo

Proszę o pytania :-)

Projektujemy bazę danych

Projektowanie bazy danych

Aby zaprojektować bazę danych,
przeprowadzamy wywiad z ekspertem z danej
dziedziny. Naszym ekspertem będzie Olga :-)

Projektowanie bazy danych



**OLGA
OWCA**

dział:	Dział Wdrożeń	
stanowisko:	szef	
ulubiona zupa:	kapuśniak	



Projektowanie bazy danych

Podczas wywiadu stosujemy regułę NAZWA-TAB:

Podkreślamy każdy rzeczownik i nazywamy
tabele tymi rzeczownikami

Można nazywać tabele rzeczownikami w liczbie
pojedynczej lub mnogiej. My będziemy nazywać
tabele rzeczownikami w liczbie pojedynczej

Projektowanie bazy danych

Jeden rzeczownik to nazwa bazy danych, czyli **Firma**. Zaczniemy od niego:

- Cześć, Olga :-) Opowiedz nam proszę o swojej firmie. Możesz zacząć, skąd chcesz :-)
- Cześć :-) W firmie pracują pracownicy.

Projektowanie bazy danych

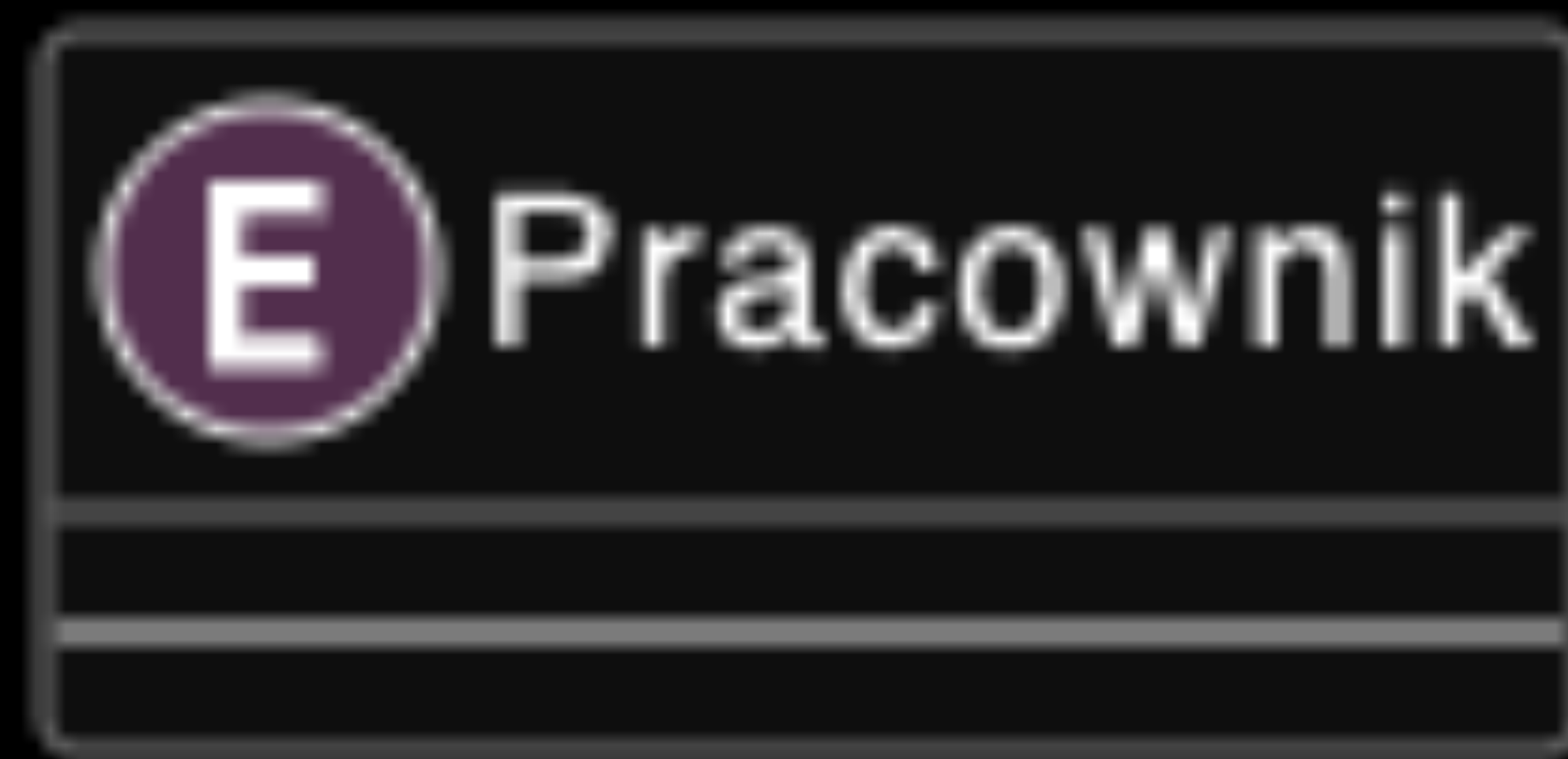
Tworzymy schemat bazy danych

Ten schemat składa się z prostokątów połączonych liniami. Po jednej stronie linii są „kurze łapki”

Projektowanie bazy danych

Zaczynamy od tabeli Pracownik

Rysujemy prostokąt. U góry tego prostokąta piszemy nazwę tabeli, czyli Pracownik



Projektowanie bazy danych

W każdej tabeli musi być klucz główny (KG)

Hm, jak nazwać ten klucz główny? Stosujemy regułę ID-KG:

Nazywamy klucz główny tabeli, łącząc skrót **id**, czyli **identyfikator**, z dopełniaczem nazwy tej tabeli. Typ tego klucza głównego to **INT**, czyli liczba całkowita

*Niektórzy nazywają klucze główne samym **id**. Nazwy z dopełniaczem nazwy tabeli są wygodniejsze. Przekonacie się o tym na następnym wykładzie :-)

Projektowanie bazy danych

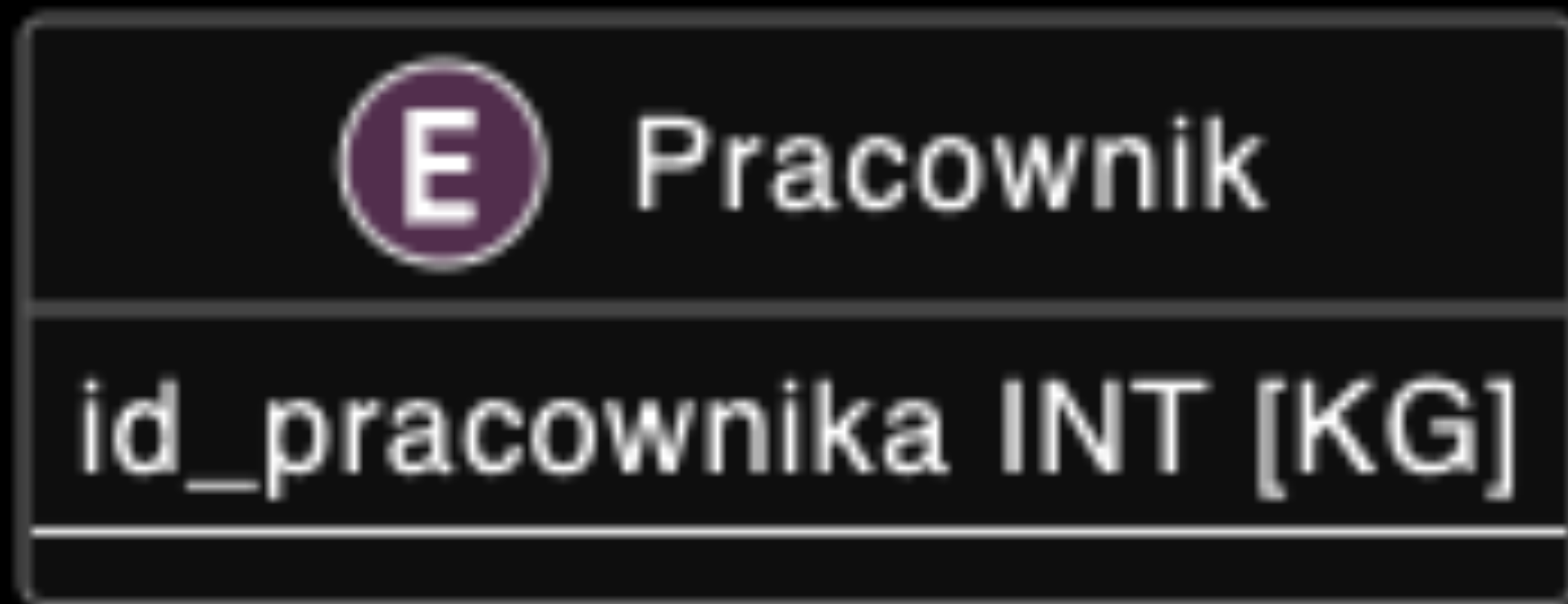
Stosujemy regułę ID-KG:

Nazywamy klucz główny tabeli, łącząc wyraz `id`, czyli `identyfikator`, z dopełniaczem nazwy tej tabeli. Typ tego klucza głównego to `INT`, czyli liczba całkowita

Dlatego klucz główny tabeli `Pracownik` nazywamy `id_pracownika`

Projektowanie bazy danych

Zapisujemy klucz główny pod nazwą tabeli
Pracownik:



Projektowanie bazy danych

Pod kluczem głównym zapisujemy pozostałe kolumny tabeli Pracownik

Hm, jakie atrybuty ma pracownik? Pracownik na pewno ma imię i nazwisko. Ich typ to TEXT

Projektowanie bazy danych

Pod kluczem głównym zapisujemy pozostałe kolumny tabeli Pracownik

 Pracownik
id_pracownika INT [KG]
imię TEXT
nazwisko TEXT

Projektowanie bazy danych

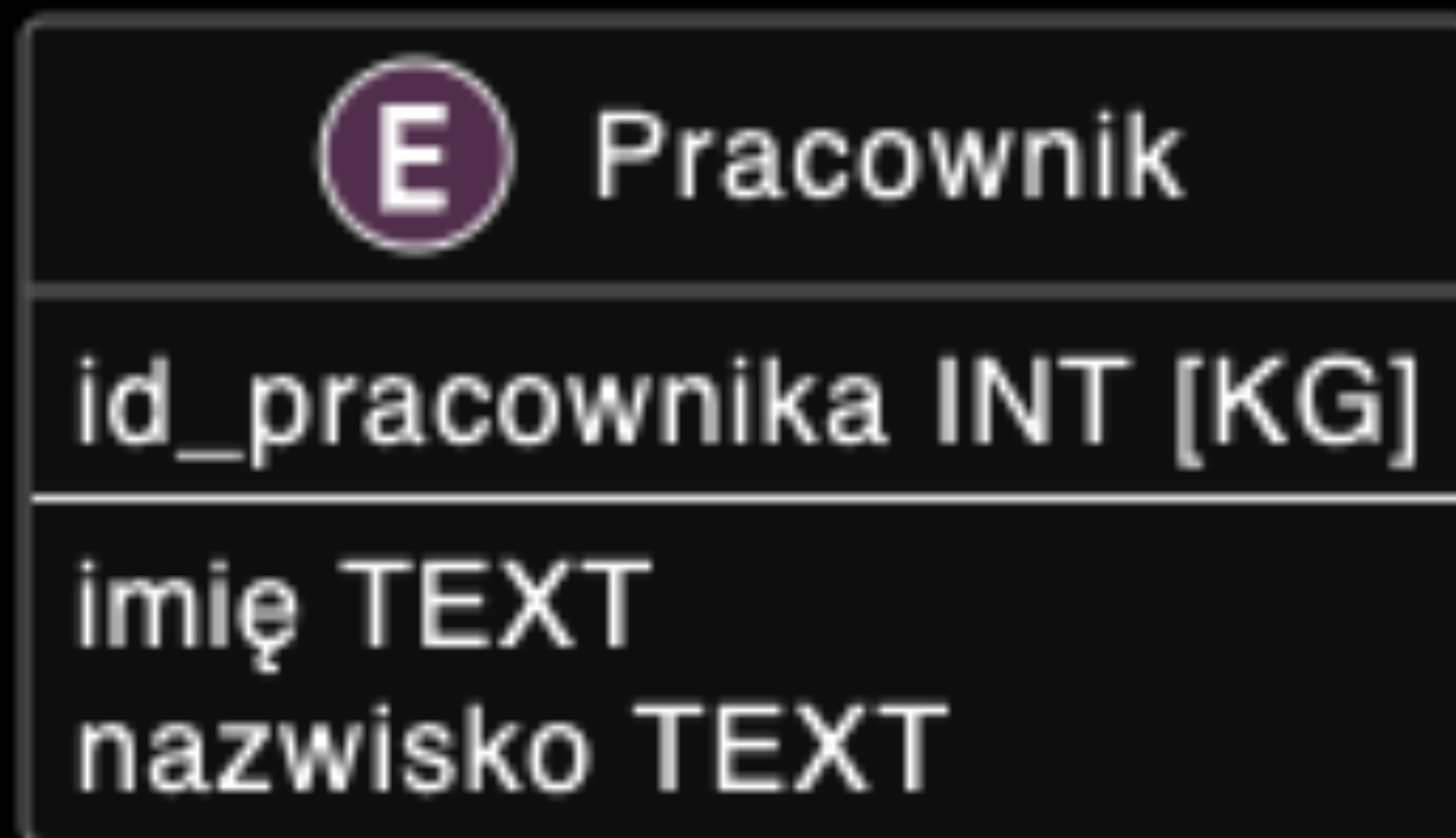
Kontynuujemy wywiad :-)

- Cześć :-) W firmie pracują pracownicy.
- Opowiedz mi o nich.
- Każdy pracownik należy do jednego działu.

Projektowanie bazy danych

Tworzymy tabelę **Dział**

Rysujemy prostokąt. U góry tego prostokąta piszemy nazwę tabeli, czyli **Dział**



Projektowanie bazy danych

W każdej tabeli musi być klucz główny (KG)

Hm, jak nazwać ten klucz główny? Znowś stosujemy regułę ID-KG:

Nazywamy klucz główny tabeli, łącząc wyraz `id`, czyli identyfikator, z dopełniaczem nazwy tej tabeli. Typ tego klucza głównego to `INT`, czyli liczba całkowita

Dlatego klucz główny tabeli `Dział` nazywamy `id_działu`

Projektowanie bazy danych

Zapisujemy klucz główny pod nazwą tabeli

Dział:

E Pracownik

id_pracownika INT [KG]

imię TEXT

nazwisko TEXT

E Dział

id_działu INT [KG]

Projektowanie bazy danych

Pod kluczem głównym zapisujemy pozostałe kolumny tabeli Dział

Hm, jakie atrybuty ma dział? Dział na pewno ma nazwę. Typ nazwy to TEXT

Projektowanie bazy danych

Pod kluczem głównym zapisujemy pozostałe kolumny tabeli **Dział**

E Pracownik

id_pracownika INT [KG]

imię TEXT

nazwisko TEXT

E Dział

id_działu INT [KG]

nazwa TEXT

Projektowanie bazy danych

Teraz się skupmy.

Każdy pracownik należy do jednego działu.
Do jednego działu może należeć wielu pracowników

To znaczy, że tabela Dział jest w relacji „jeden do wielu” z tabelą Pracownik

Projektowanie bazy danych

Tabela Dział jest w relacji „jeden do wielu” z tabelą Pracownik

Relację „jeden do wielu” między dwiema tabelami oznaczamy, łącząc linią prostokąty odpowiadające tym tabelom. Po stronie „wielu” rysujemy „kurzą łapkę” :-)

Projektowanie bazy danych

Relacja „jeden do wielu” między dwiema tabelami zachodzi wtedy, gdy klucz obcy po stronie „wielu” odpowiada kluczowi głównemu po stronie „jeden”

Aby nazwać ten klucz obcy, stosujemy regułę
1-KG-WIELE-KO: Gdy tworzymy relację „jeden do wielu”, nadajemy KO tabeli po stronie „wielu” tę samą nazwę, którą ma KG tabeli po stronie „jeden”

Projektowanie bazy danych

Aby nazwać klucz obcy, stosujemy regułę
1-KG-WIELE-KO: Gdy stworzymy relację „jeden do wielu”, nadajemy KO tabeli po stronie „wielu” tę samą nazwę, którą ma KG tabeli po stronie „jeden”

Dlatego nadajemy kluczowi obcemu tabeli Pracownik nazwę `id_działu`. Tak jest wygodnie.
Przekonacie się o tym na następnym wykładzie :-)

Projektowanie bazy danych

Nadajemy kluczowi obcemu tabeli Pracownik nazwę `id_działu`



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Każdy pracownik należy do jednego działu.
- Opowiedz mi o działach.
- Każdy dział ma szefa.
- Czy szef to też pracownik?
- Tak :-)

Projektowanie bazy danych

Ponieważ szef działu to też pracownik,
trzeba dodać do tabeli **Dział** klucz obcy,
który odpowiada kluczowi głównemu
id_pracownika tabeli **Pracownik**

Aby nazwać ten klucz obcy, stosujemy regułę
NAZWA-KO

Projektowanie bazy danych

Aby nazwać klucz obcy, stosujemy regułę NAZWA-K0:

- + Tworzymy relację „jeden do wielu”. Po stronie „wielu” stoi ta tabela, w której tworzymy K0
- + Nazywamy tę relację, łącząc mianownik (kto? co?) rzeczownika, który określa funkcję K0, z dopełniaczem (kogo? czego?) nazwy tabeli po stronie „wielu”
- + Nazywamy ten K0, łącząc nazwę KG tej tabeli, która go zawiera, z dopełniaczem nazwy utworzonej relacji

Projektowanie bazy danych

Dlatego:

- + Tworzymy relację „jeden do wielu” z tabelą Pracownik po stronie „jeden” i tabelą Dział po stronie „wielu”
- + łączymy mianownik (kto?) rzeczownika „szef” z dopełniaczem (czego?) nazwy tabeli Dział, otrzymując nazwę relacji „szef działu”
- + łączymy nazwę KG tabeli Pracownik, czyli id_pracownika, z dopełniaczem (czego?) nazwy relacji „szef działu”, otrzymując nazwę KO tabeli Dział:
id_pracownika_szefa_działu

Projektowanie bazy danych



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Czy szef to też pracownik?
- Tak :-)
- Czym się zajmuje pracownik?
- Pracownik pracuje nad zadaniami.

Projektowanie bazy danych

Tworzymy tabelę Zadanie

W każdej tabeli musi być klucz główny. Zgodnie z regułą ID-KG nazywamy KG tabeli Zadanie `id_zadania`. Typ tego klucza głównego to `INT`

Pod kluczem głównym zapisujemy pozostałe kolumny tabeli Zadanie. Hm, jakie atrybuty ma zadanie? Zadanie na pewno ma nazwę :-) Typ nazwy to `TEXT`

Projektowanie bazy danych



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Pracownik pracuje nad zadaniami.
- Opowiedz mi o nich.
- Jedno zadanie przypada tylko jednemu zespołowi i na odwrót: jeden zespół zajmuje się tylko jednym zadaniem.

Projektowanie bazy danych

Hm, to znaczy, że zadania i zespoły są w relacji „jeden do jednego”

Tworzymy tabelę Zespół. Zgodnie z regułą ID-KG nazywamy KG tabeli Zespół `id_zespołu`. Typ tego klucza głównego to INT

Projektowanie bazy danych



Projektowanie bazy danych

Hm, czy zespół ma jakieś atrybuty? Kontynuujmy wywiad :-)

– Jedno zadanie przypada tylko jednemu zespołowi i na odwrót: jeden zespół zajmuje się tylko jednym zadaniem.

– Czy zespoły mają nazwy?

– Nie, zespoły nie mają nazw.

Projektowanie bazy danych

Wróćmy do naszego schematu. Jedno zadanie przypada tylko jednemu zespołowi i na odwrót: jeden zespół zajmuje się tylko jednym zadaniem, to znaczy, że zadania i zespoły są w relacji „jeden do jednego”

Gdy dwie tabele są w relacji „jeden do jednego”, scalałyśmy te dwie tabele, bo tylko jedna z tych tabel jest potrzebna

Projektowanie bazy danych

Hm, ale którą tabelę zostawić? Ponieważ firma składa się z ludzi, a ludzie tworzą zespół, zostawiamy tabelę Zespół. Hm, ale co zrobić z kolumnami tabeli Zadanie? Klucz główny tabeli Zadanie już nie jest potrzebny. Zmieniamy nazwę kolumny nazwa na nazwa_zadania i przenosimy tę kolumnę do tabeli Zespół. Tabela Zadanie już nie jest potrzebna :-)

Projektowanie bazy danych



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Nie, zespoły nie mają nazw.
- Czy każdy pracownik należy do jednego zespołu?
- Nie :-) Pracownik może należeć do wielu zespołów i na odwrót: jeden zespół może się składać z wielu pracowników.

Projektowanie bazy danych

Hm, to znaczy, że pracownicy i zespoły są w relacji „wiele do wielu”

Nie da się bezpośrednio odwzorować w bazie danych relacji „wiele do wielu”. Trzeba utworzyć tabelę pośrednią. Ta tabela pośrednia będzie w relacji „jeden do wielu” z tabelą Pracownik i z tabelą Zespół

Projektowanie bazy danych

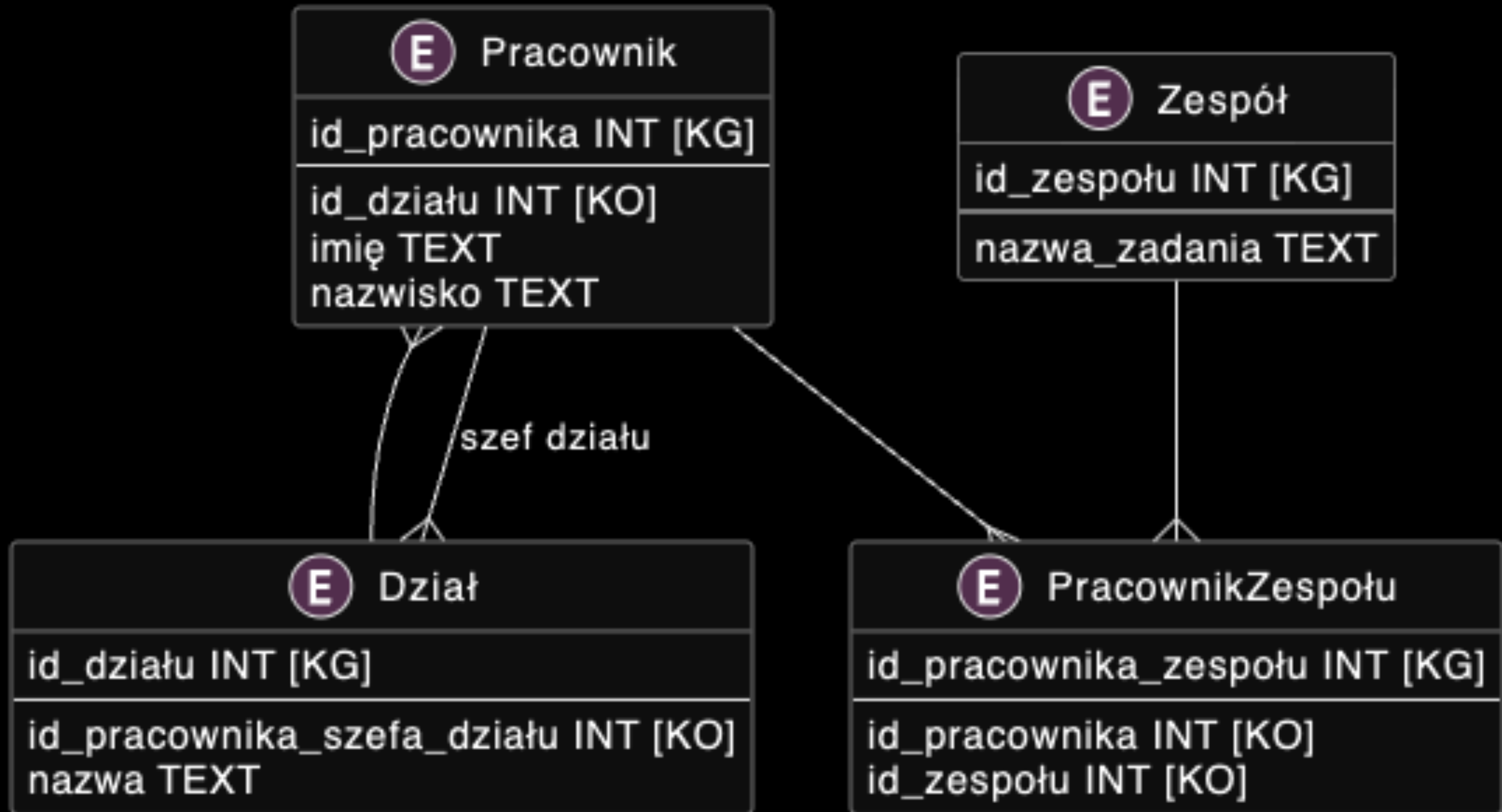
Hm, tylko jak nazwać tę tabelę pośrednią?

Zastosujmy regułę NAZWA-TAB-POŚR: Gdy tworzymy tabelę pośrednią, łączymy nazwy tych tabel, które łączy ta tabela pośrednia. Jedną z tych nazw zapisujemy w mianowniku, a drugą w dopełniaczu

łącząc nazwy tabel Pracownik i Zespół, otrzymujemy nazwę PracownikZespołu albo ZespółPracownika.

Wybieramy pierwszą nazwę: PracownikZespołu

Projektowanie bazy danych



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Nie :-) Pracownik może należeć do wielu zespołów i na odwrót: jeden zespół może się składać z wielu pracowników.
- Opowiedz mi więcej o zespołe.
- Każdy zespół ma kierownika. Kierownik to też pracownik :-)

Projektowanie bazy danych

Ponieważ kierownik zespołu to też pracownik, trzeba dodać do tabeli Zespół klucz obcy, który odpowiada kluczowi głównemu `id_pracownika` tabeli Pracownik

Aby nazwać ten klucz obcy, znów stosujemy regułę NAZWA-KO

Projektowanie bazy danych

Aby nazwać klucz obcy, stosujemy regułę NAZWA-K0:

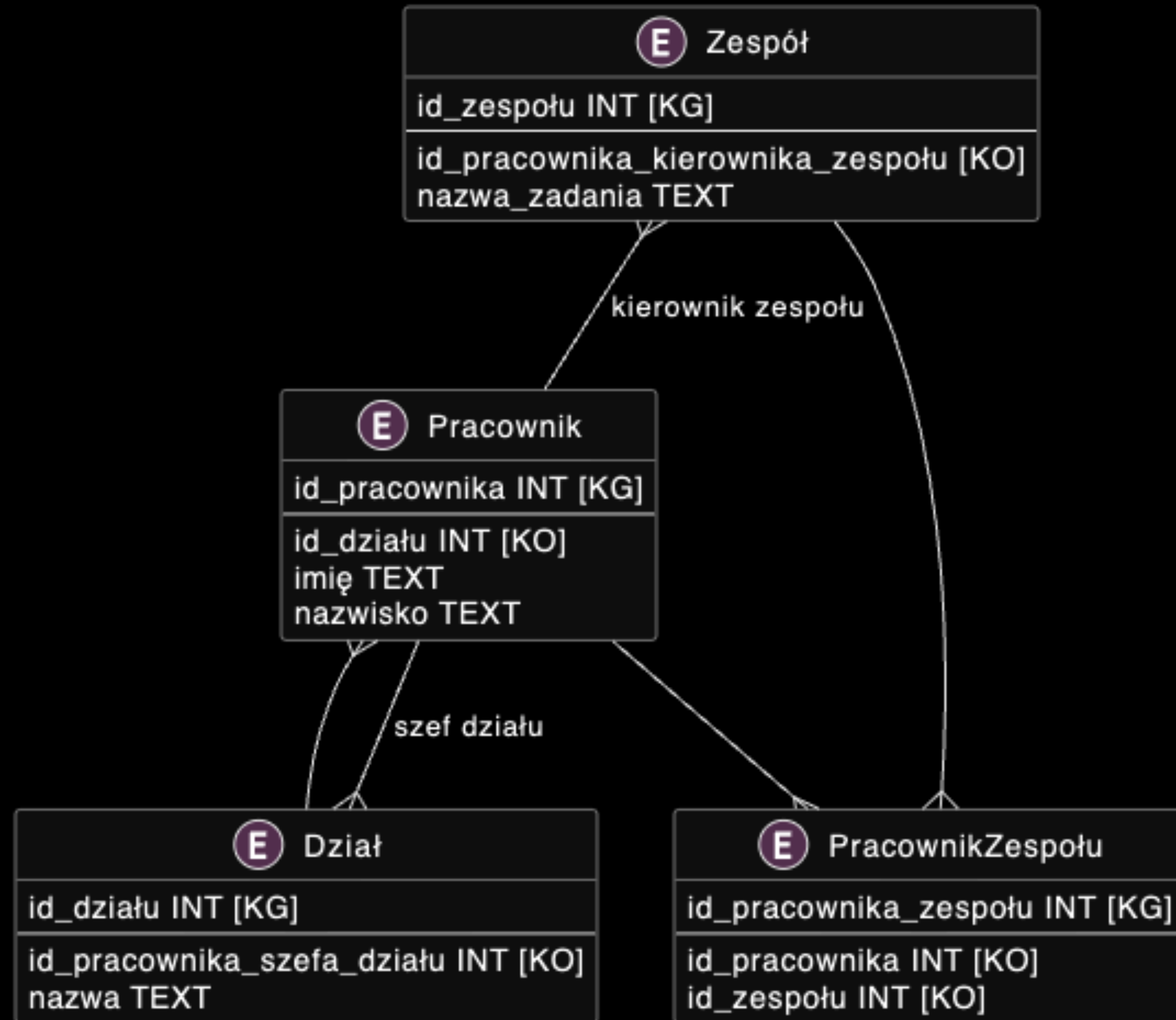
- + Tworzymy relację „jeden do wielu”. Po stronie „wielu” stoi ta tabela, w której tworzymy K0
- + Nazywamy tę relację, łącząc mianownik (kto? co?) rzeczownika, który określa funkcję K0, z dopełniaczem (kogo? czego?) nazwy tabeli po stronie „wielu”
- + Nazywamy ten K0, łącząc nazwę KG tej tabeli, która go zawiera, z dopełniaczem nazwy utworzonej relacji

Projektowanie bazy danych

Dlatego:

- + Tworzymy relację „jeden do wielu” z tabelą Pracownik po stronie „jeden” i tabelą Zespół po stronie „wielu”
- + łączymy mianownik (kto?) rzeczownika „kierownik” z dopełniaczem (czego?) nazwy tabeli Zespół, otrzymując nazwę relacji „kierownik zespołu”
- + łączymy nazwę KG tabeli Pracownik, czyli id_pracownika, z dopełniaczem (czego?) nazwy relacji „kierownik zespołu”, otrzymując nazwę KO tabeli Zespół: id_pracownika_kierownika_zespołu

Projektowanie bazy danych



Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Każdy zespół ma kierownika. Kierownik to też pracownik :-)
- Opowiedz mi o kierownikach.
- Każdy kierownik ustala wypłatę dla pracowników zespołu.

Projektowanie bazy danych

Tworzymy tabelę Wypłata. Hm, jakie atrybuty ma wypłata? Wypłata na pewno ma kwotę. Zastosujmy regułę NAZWA-MIAR: Do nazw wszystkiego, co można mierzyć na różne sposoby, dodajemy jednostkę miary

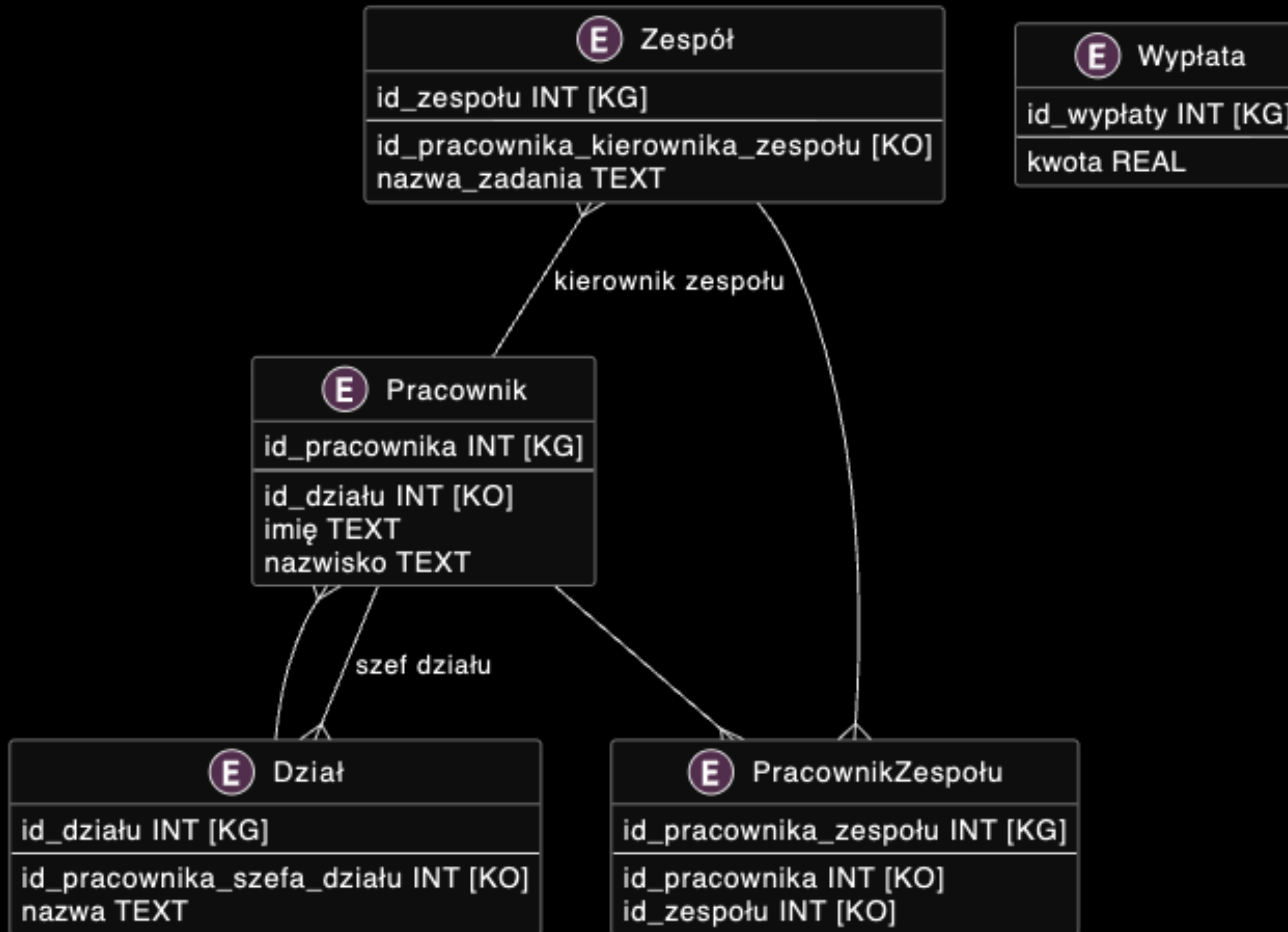
*Twórcy sondy Mars Climate Orbiter nie stosowali tej reguły. Zespół, który tworzył program kontroli naziemnej, używał funtów jako jednostki siły. Zespół, który tworzył program sondy, używał niutonów jako jednostki siły. W wyniku tego nieporozumienia sonda spaliła się w atmosferze Marsa

Projektowanie bazy danych

Tworzymy tabelę Wypłata. Hm, jakie atrybuty ma wypłata? Wypłata na pewno ma kwotę. Zastosujemy regułę NAZWA-MIAR: Do nazw wszystkiego, co można mierzyć na różne sposoby, dodajemy jednostkę miary

Dlatego nazwiemy tę kolumnę kwota_PLN.
Ponieważ kwoty wypłat w PLN mogą mieć grosze po przecinku, typ kolumny kwota_PLN to REAL

Projektowanie bazy danych



Projektowanie bazy danych

- Co zrobimy, jeśli kiedyś wypłaty będą w euro?
- O tym dowiemy się na następnym wykładzie :-)

Projektowanie bazy danych

Kontynuujemy wywiad :-)

- Każdy kierownik ustala wypłatę dla pracowników zespołu.
- Opowiedz mi o wypłatach.
- Każda wypłata następuje, kiedy zespół kończy zadanie.
- Czyli każdy pracownik zespołu dostaje jedną wypłatę?
- Tak :-)

Projektowanie bazy danych

Hm, to znaczy, że tabele PracownikZespołu i Wypłata są w relacji „jeden do jednego”

Gdy dwie tabele są w relacji „jeden do jednego”, scalałyśmy te dwie tabele, bo tylko jedna z tych tabel jest potrzebna

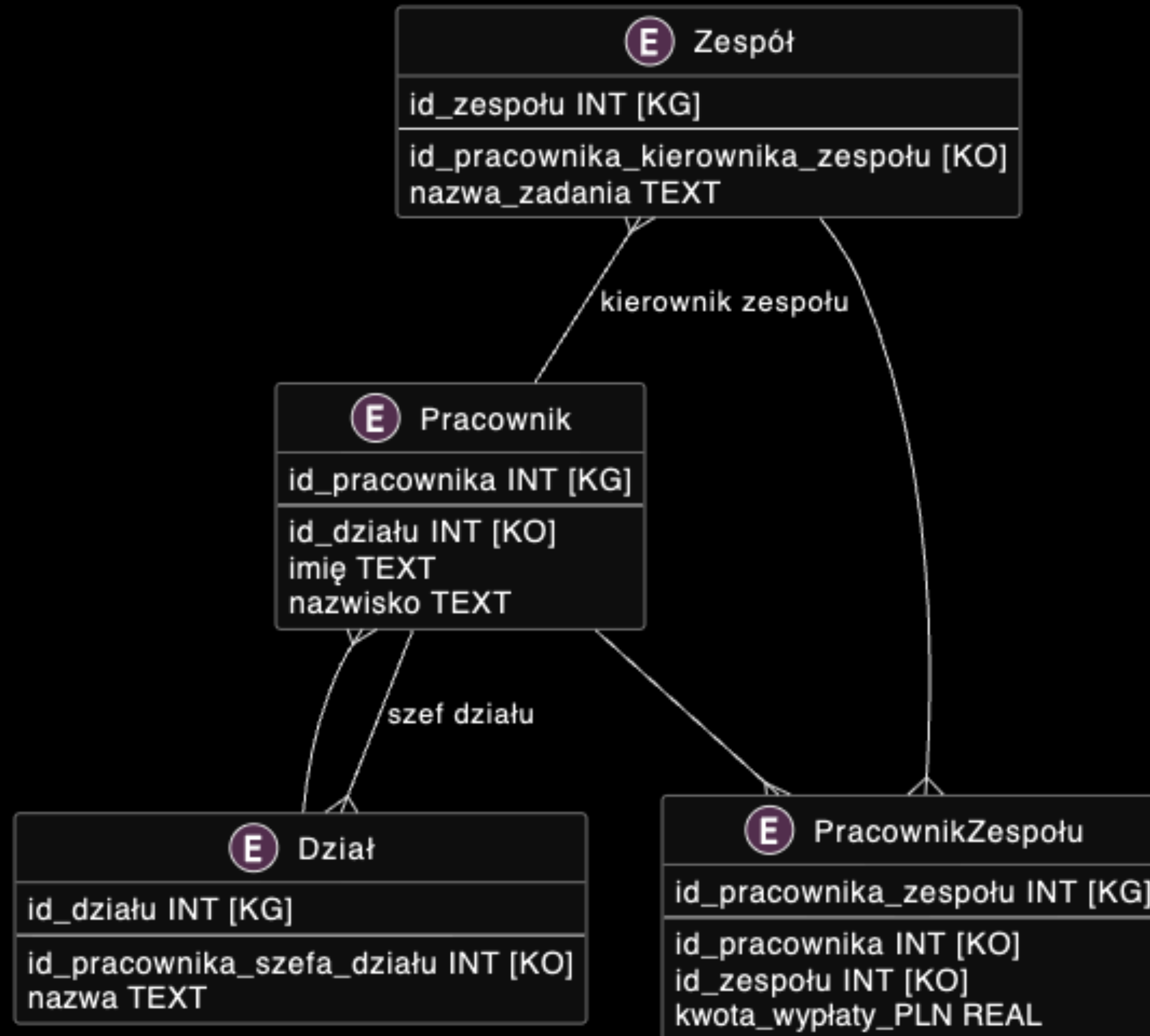
Hm, ale którą tabelę zostawić? Ponieważ firma składa się z pracowników, a nie z wypłat, zostawiamy tabelę PracownikZespołu

Projektowanie bazy danych

Hm, ale co zrobić z kolumnami tabeli Wypłata?

Klucz główny tabeli Wypłata już nie jest potrzebny. Zmieniamy nazwę kolumny kwota_PLN na kwota_wypłaty_PLN i przenosimy tę kolumnę do tabeli PracownikZespołu. Tabela Wypłata już nie jest potrzebna :-)

Projektowanie bazy danych

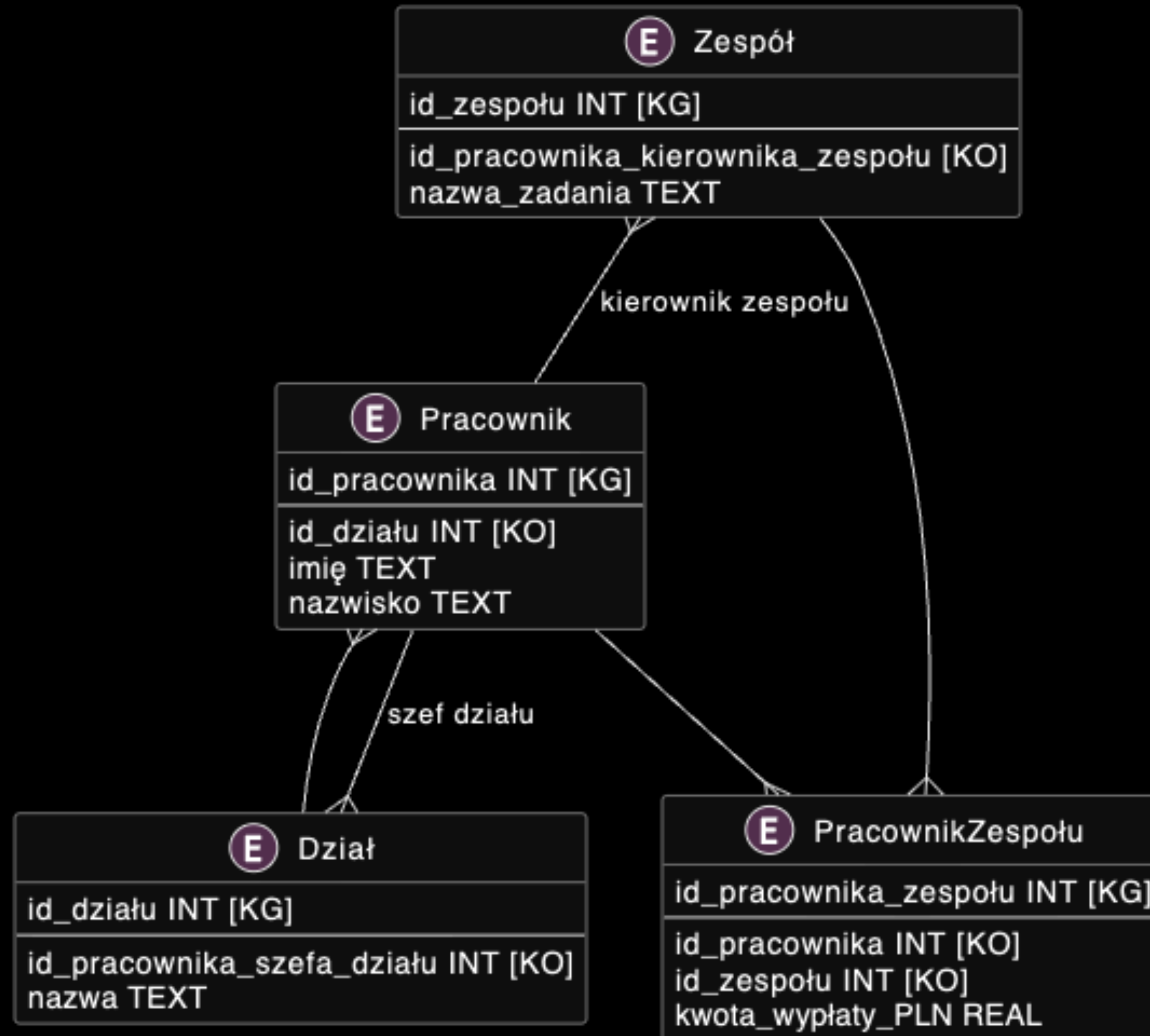


Projektowanie bazy danych

Kontynuujmy wywiad :-)

- Czyli każdy pracownik zespołu dostaje jedną wypłatę?
- Tak :-)
- Czy chciałabyś coś jeszcze dodać?
- Nie :-)
- Dziękuję ci za wywiad.

Projektowanie bazy danych



Projektowanie bazy danych

Tej bazy danych już się nie da uprościć :-)

Powtórzmy te reguły, które stosowaliśmy, aby nazywać tworzone tabele, relacje i kolumny

Projektowanie bazy danych

Reguła NAZWA-TAB:

Podkreślamy każdy rzeczownik i nazywamy
tabele tymi rzeczownikami

Projektowanie bazy danych

Reguła ID-KG:

Nazywamy klucz główny tabeli, łącząc wyraz **id**, czyli **identyfikator**, z dopełniaczem nazwy tej tabeli. Typ tego klucza głównego to **INT**, czyli liczba całkowita

Projektowanie bazy danych

Reguła 1-KG-WIELE-KO:

Gdy tworzymy relację „jeden do wielu”,
nadajemy KO tabeli po stronie „wielu”
tę samą nazwę, którą ma KG tabeli po stronie
„jeden”

Projektowanie bazy danych

Reguła NAZWA-KO:

- + Tworzymy relację „jeden do wielu”. Po stronie „wielu” stoi ta tabela, w której tworzymy KO
- + Nazywamy tę relację, łącząc mianownik (kto? co?) rzeczownika, który określa funkcję KO, z dopełniaczem (kogo? czego?) nazwy tabeli po stronie „wielu”
- + Nazywamy ten KO, łącząc nazwę KG tej tabeli, która go zawiera, z dopełniaczem nazwy utworzonej relacji

Projektowanie bazy danych

Reguła NAZWA-TAB-POŚR:

Gdy tworzymy tabelę pośrednią, łączymy nazwy tych tabel, które łączy ta tabela pośrednia. Jedną z tych nazw zapisujemy w mianowniku, a drugą w dopełniaczu

Projektowanie bazy danych

Reguła NAZWA-MIAR:

Do nazw wszystkiego, co można mierzyć
na różne sposoby, dodajemy jednostkę miary

Proszę o pytania :-)

Jak wpisywać polecenia
SQL-a

Jak wpisywać polecenia SQL-a

Wewnątrz poleceń SQL-a możemy pisać małe lub duże litery i tyle spacji i nowych linii, ile chcemy – poza tekstami, bo w tekstach każdy znak ma znaczenie :-)

Po każdym poleceniu SQL-a stawiamy średnik;

Jak wpisywać polecenia SQL-a

W komentarzach umieszczamy notatki dla czytelnika kodu lub „wyłączamy” fragmenty kodu. System zarządzania bazą danych ignoruje komentarze

-- To jest komentarz jednoliniowy

/* Taki komentarz może
zajmować więcej linii */

Jak wpisywać polecenia SQL-a

Te polecenia znaczą to samo:

```
SELECT  
    nr_indeksu,  
    imię,  
    nazwisko  
FROM Student;
```

```
select nr_indeksu, imię, nazwisko from student;
```

```
SeLeCt Nr_InDeKsU, ImIe, NaZwIsKo FrOm StUdEnT;
```

Tworzymy bazę danych

Tworzymy bazę danych

Aby utworzyć tabelę w SQL-u, piszemy `CREATE TABLE`, a potem piszemy nazwę tabeli i listę kolumn wraz z ich typami danych, otoczona nawiasami okrągłymi. Pomiedzy każdą parą sąsiednich kolumn wraz z ich typami danych należy postawić przecinek

```
CREATE TABLE Student (  
    nr_indeksu INT PRIMARY KEY,  
    imię TEXT,  
    nazwisko TEXT  
);
```

Gdy kolumna zawiera klucz główny, piszemy po jej typie danych `PRIMARY KEY`

Tworzymy bazę danych

Gdy kolumna zawiera klucz obcy, możemy napisać po jej typie danych

REFERENCES InnaTabela(klucz_główny_innej_tabeli)

```
CREATE TABLE Pracownik (  
    id_pracownika INT PRIMARY KEY,  
    id_działu INT REFERENCES Dział(id_działu),  
    imię TEXT,  
    nazwisko TEXT  
);
```


Tworzymy bazę danych

Tak tworzymy tabelę Dział:

```
CREATE TABLE Dział (  
    id_działu INT PRIMARY KEY,  
    id_pracownika_szefa_działu INT  
        REFERENCES Pracownik(id_pracownika),  
    nazwa TEXT  
);
```

Tworzymy bazę danych

Tak tworzymy tabelę Zespół:

```
CREATE TABLE Zespół (  
    id_zespołu INT PRIMARY KEY,  
    id_pracownika_kierownika_zespołu INT  
        REFERENCES Pracownik(id_pracownika),  
    nazwa_zadania TEXT  
);
```

Tworzymy bazę danych

Tak tworzymy tabelę pośrednią PracownikZespołu:

```
CREATE TABLE PracownikZespołu (  
    id_pracownika_zespołu INT PRIMARY KEY,  
    id_pracownika INT  
        REFERENCES Pracownik(id_pracownika),  
    id_zespołu INT  
        REFERENCES Zespół(id_zespołu),  
    kwota_wypłaty_PLN REAL  
);
```


Stałe różnych typów

Stałe różnych typów

+ TEXT — Tekst

+ INT — Liczby całkowite

+ REAL — Liczby rzeczywiste

Stałe różnych typów

Aby zapisać tekst w SQL-u, otaczamy ten tekst pojedynczymi cudzysłowami, czyli apostrofami. Tak zapisujemy tekst Chrząszcz brzmi w trzcinie:

'Chrząszcz brzmi w trzcinie'

Aby zapisać w SQL-u taki tekst, który zawiera apostrofy, piszemy dwa apostrofy obok siebie. Tak zapisujemy tekst rock'n'roll:

'Rock''n''roll'

Stałe różnych typów

Tak zapisujemy liczby całkowite w SQL-u:

-14

125

Stałe różnych typów

Aby zapisać liczbę rzeczywistą w SQL-u, piszemy zamiast przecinka dziesiętnego kropkę dziesiętną. Tak zapisujemy liczbę minus jedna czwarta:

-0.25

Aby zapisać w SQL-u bardzo duże lub bardzo małe liczby, korzystamy z notacji naukowej. Tak zapisujemy liczbę Avogadra $6,02214076 \times 10^{23} \text{ [mol}^{-1}\text{]}$:

6.02214076e23 lub 6.02214076E23

Wypełniamy bazę danych
danymi

Wypełniamy bazę danych danymi

Aby dodać wiersz do tabeli, używamy instrukcji `INSERT INTO`

Gdy dodajemy dane do wybranych kolumn, piszemy

```
INSERT INTO NazwaTabeli (kolumna1, kolumna2)  
VALUES (wartość1, wartość2);
```


Wypełniamy bazę danych danymi

Na przykład:

```
INSERT INTO Pracownik (  
    id_pracownika, imię, nazwisko)  
VALUES (1, 'Olga', 'Owca');
```

Wypełniamy bazę danych danymi

Możemy też dodać do tabeli wiele wierszy naraz, na przykład:

```
INSERT INTO Pracownik (  
    id_pracownika, imię, nazwisko)  
VALUES  
    (2, 'Wiktor', 'Wilk'),  
    (3, 'Kasia', 'Kot'),  
    (4, 'Lucyna', 'Lis');
```

Wypełniamy bazę danych danymi

Gdy dodajemy dane do wszystkich kolumn, możemy nie wymieniać nazw tych kolumn

Trzeba wtedy podać wartości w tej samej kolejności, w jakiej zostały utworzone kolumny

```
INSERT INTO NazwaTabeli  
VALUES (wartość1, wartość2, wartość3);
```


Wypełniamy bazę danych danymi

Na przykład:

```
INSERT INTO Dział VALUES
-- id_działu, nazwa, id_pracownika_szefa_działu
  (1, 'Dział Analiz', 3),    -- szef: Kasia
  (2, 'Dział Wdrożeń', 1);  -- szef: Olga
```

Pierwsze zapytanie
do bazy danych

Pierwsze zapytanie do bazy danych

Polecenie

```
SELECT * FROM NazwaTabeli;
```

pobiera dane z wszystkich kolumn i wszystkich wierszy tabeli

Pierwsze zapytanie do bazy danych

Na przykład:

```
SELECT * FROM Pracownik;
```

id_pracownika	id_działu	imię	nazwisko
1	NULL	Olga	Owca
2	NULL	Wiktor	Wilk
3	NULL	Kasia	Kot
4	NULL	Lucyna	Lis

Hm, żaden pracownik nie należy do żadnego działu :-
To nic, poprawimy to na następnym wykładzie :-)

To już koniec :-)

Tematy dzisiejszego wykładu

Z tego wykładu dowiedzieliście się:

- + o wartości **NULL**
 - + o typach danych
 - + o relacjach „jeden do wielu”
 - + jak zaprojektować relacyjną bazę danych
 - + jak utworzyć bazę danych
 - + jak wypełnić bazę danych danymi
- ...i poznacie pierwsze zapytanie :-)

Proszę o pytania :-)

Zapraszam was na następny wykład. Dowiedzie się z niego:

+ o klauzuli WHERE

+ o tym, jak zmieniać bazę danych instrukcją ALTER TABLE i UPDATE

+ o tym, jak zadawać zapytania ze złączeniami, czyli z klauzulą JOIN

Podziękowania

Podziękowania @+→-

Dziękuję:

- + prof. Stanisławowi Koziełskiemu
- + Ireneuszowi Morawskiemu
- + Piotrowi Wieczorkowi
- + Annie Zając

Źródła grafik

Źródła grafik

+ [https://commons.wikimedia.org/wiki/
File:Z%C3%BCnd_Eichenwald_1882.jpg](https://commons.wikimedia.org/wiki/File:Z%C3%BCnd_Eichenwald_1882.jpg)

+ zbiory własne

+ <https://gemini.google.com/>